TRINITY COLLEGE · NOVEMBER 2, 2026

AI Integration

Team Project 1 presentations, then the program listens and speaks

Ken Kousen · CPSC 415
Monday 1:30–4:10 · MC-225

This afternoon

1:35–2:35	**Four presentations**, 10 minutes + 5 of questions, order drawn now
During each team's questions	the other teams fill in the **peer notes form** (link on Moodle and the Week 7 page)
2:35–2:45	What the room saw
3:00–3:40	**Audio:** speech in, speech out, the transcript checked in code
3:40–3:50	**Hosting:** where the key lives, and the proof
3:50–4:10	The lab (due Nov 9), questions

Every member explains every part. Expect a question to the member who did not write the piece on screen.

Presentations: what to show

- The app **running**, one use case end to end. Localhost is fine.
- One **artifact** from the chain on screen: the intent, the spec's model decision, the plan's Proof, a PR with the reviewer's findings.
- One thing you **tried that failed**, or took out. What you did about it.
- The chain exists: intent, spec, plan, tests, a reviewed PR **per member**, `ANNOTATION.md`.

> Ten minutes. Then five of questions, and a question goes to whoever did not write the part we are looking at.

The peer notes form

1. One thing this team did that you would **borrow** for your own project, and why.
2. One question you would ask them in a **Comprehension Defense**: something you would need explained before you believed it works.
3. One claim you were **not convinced by**, and what evidence would convince you.

- Once per presenting team, during their question time.
- Your name is recorded, not shown to them.
- Each team gets the notes written about them.
- Every row counts toward participation.

What the room saw

- Which critique paths actually read the image, and which only read the prompt?
- Where did a second model disagree with the first, and who was right?
- Which PR findings were real, and which were the reviewer being confidently wrong?
- What did the projects cost, and what was the most expensive single call?

Two more request shapes

Speech → text

`POST /api/v1/audio/transcriptions`
`model`, `input_audio: {data, format}` in
JSON out: `text`, `usage.seconds`, `usage.cost`
priced **per second** of audio (or per token)

Text → speech

`POST /api/v1/audio/speech`
`model`, `input`, `voice`, `response_format` in
audio bytes out, `X-Generation-Id` header, no usage
priced **per character** of text

- Same key, same base URL, as chat (Week 2), embeddings (Week 4), images (Week 6).
- `GET /api/v1/models?output_modalities=transcription` and `=speech` list what each endpoint can call, with prices and voices.

Raw, and through the SDK

```
b64 = base64.b64encode(open("q.wav","rb").rea
body = {"model": STT, "input_audio":
        {"data": b64, "format": "wav"}}
req = urllib.request.Request(
  "https://openrouter.ai/api/v1/audio/transcr
  json.dumps(body).encode(),
  {"Authorization": f"Bearer {KEY}",
   "Content-Type": "application/json"})
d = json.load(urllib.request.urlopen(req))
print(d["text"], d["usage"]["cost"])
```

```
r = client.stt.create_transcription(
  model=STT,
  input_audio={"data": b64,
               "format": "wav"})
print(r.text, r.usage.cost)

r = client.tts.create_speech(
  model=TTS, input=text,
  voice=VOICE, response_format="mp3")
data = r.read()      # bytes
```

- The speech reply is a stream: `read()` it, check the content type is `audio/…` **before** saving, or you will save a JSON error as `.mp3`.

Live E: one sentence, six voices

```
python3 speak.py "Welcome to CPSC 415. Today the program listens, and then
python3 speak.py "..." --voice en-GB-Emily:MAI-Voice-2.1-Flash
python3 speak.py "..." --voice en-IN-Priya:MAI-Voice-2.1-Flash
python3 speak.py "Bienvenidos a CPSC 415. ..." --voice es-MX-Valeria:MAI-Vo
python3 speak.py "सीपीएससी 415 में आपका स्वागत है। ..." --model x-ai/grok-voice-tt
python3 speak.py "..." --model elevenlabs/eleven-v4-turbo --voice george
```

Live E: what six voices cost

Model	Per character	67 characters	Time
`microsoft/mai-voice-2.1-flash` (class)	$0.000015	**$0.0010**	0.5–0.8 s
`x-ai/grok-voice-tts-1.0`	$0.000015	$0.0011	0.9 s
`elevenlabs/eleven-v4-turbo`	$0.000040	$0.0027	1.5–2.1 s
`hexgrad/kokoro-82m` (open weights)	$0.0000006	$0.00004	0.4 s … 197 s

Live F: a clip, transcribed, and checked

```
clip-02.wav: ok  wer=0.0  missing=[]  6.45s audio  $0.0000215  0.7s
    got: "Have you ever deployed anything with Kubernetes? Be honest, I'm c
```

- The transcript is **model output**. The code cannot hear; it can measure.
- **Word error rate** against the text you wrote down first: edits per expected word.
- **Words that must survive:** your name, the course number. "boot camp" counts as "bootcamp"; "Cousin" does not count as "Kousen".
- An **empty** reply is a failed transcript, not a crash.

```
openai/whisper-large-v3-turbo   $0.0000033 per second    6 s clip ≈ $0.0000
elevenlabs/scribe-v2            $0.00006  per second     6 s clip ≈ $0.0004
```

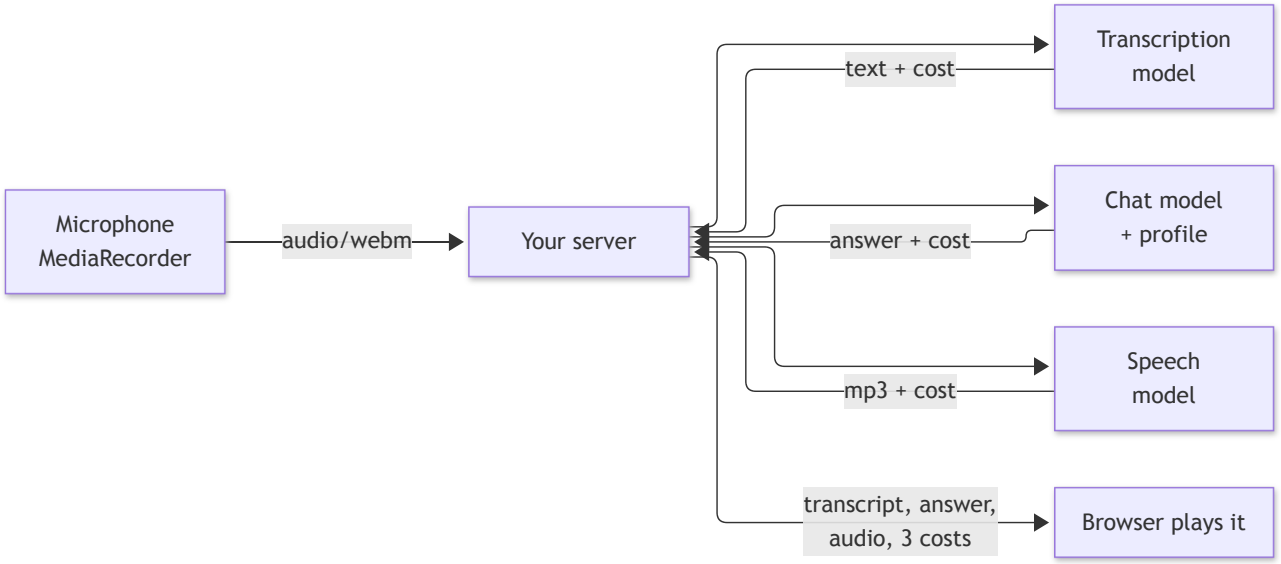
The transcript that stopped halfway

```
clip-02.mp3 (Kokoro):  whisper  3.1 s audio  "Have you ever deployed anythi
clip-02.mp3 (Kokoro):  scribe   6.5 s audio  "... I'm comparing you with a
```

- Two cheap speech models wrote MP3s whose **header says the wrong length**. Whisper's host trusted the header: `usage.seconds` matched the header, not the audio.
- Scribe decoded the audio and read all of it.
- The fix is not a better model: ask for **PCM** and write the WAV yourself (`wave`, standard library). No header, nothing to be wrong.

> Both models were fine. The pair failed. The second transcriber is what told us.

The loop


Microphone
MediaRecorder
audio/webm
Your server
Transcription
model
text + cost
Chat model
+ profile
answer + cost
Speech
model
mp3 + cost
transcript, answer,
audio, 3 costs
Browser plays it

The loop: what a turn costs

- Three billed calls per turn. On the class models: transcription ≈ $0.00002, answer ≈ $0.0005, speech ≈ $0.003 (two or three sentences at $0.000015 a character). **About half a cent a turn**, measured $0.0035–0.0057.
- The answer function is the **same one** the typed path uses. The refuse-to-fabricate rule applies to what is said aloud.
- The browser sends audio and gets audio. It never sees the key or the model names.

A third door: audio in a chat message

```
client.chat.send(model=AUDIO_CHAT_MODEL, messages=[{"role": "user", "conten
    {"type": "text", "text": "What did the speaker ask, and in what tone?"},
    {"type": "input_audio", "input_audio": {"data": b64, "format": "wav"}}]
```

You want	Use	You get
the words	`/audio/transcriptions`	text + seconds + cost
a model to **reason about** the audio: tone, a question, audio + image	`input_audio` part on `/chat/completions`	a chat completion
the words spoken	`/audio/speech`	bytes

Hosting: one rule, three hosts

Host	Your function	Where the key lives
Cloudflare Pages (course default)	`functions/api/ask.js` → `/api/ask`, `env.OPENROUTER_API_KEY`	`wrangler pages secret put`, or the dashboard, encrypted
Vercel	`api/ask.js` → `/api/ask`, `process.env.…`	Project settings, sensitive
here.now	no code: a **proxy route** in `.herenow/proxy.json`	a **service variable** injected into the proxied request
Your own server	the Week 4 server in a container	the host's environment

Hosting: the key and the cap

- The site gets **its own key with a credit limit**. A voice turn is about half a cent; a $5 limit is a thousand of them, and when it leaks you rotate one key.
- Cap the route: message length, a rate limit per IP, one model call per request, CORS to your own origin.

Hosting: the proof

1. **View source** of the deployed page: no key, no model name.
2. **Network tab:** the only request leaving the browser is to your `/api/ask`, with no `Authorization` header. The provider never appears.
3. **grep** the build output and the history for `sk-or`: nothing. (A key in any commit is compromised: rotate.)
4. **The Week 5 browser check against the live URL.** Its last three assertions are the deployment's acceptance test.

> A site that passes all four and answers one question from the test set is deployed. Before November 30.

Lab: hear, think, speak

0 Portfolio repo, `voice/`, SDK in `requirements.txt`, rules

1 Intent and spec: transcriber, speaker

2 Plan, proof split, with the cost

3 Tests first, then the checker

4 **Your own voice,** two clips, two transcribers, ~$0.002

5 The answer spoken, two voices, ~$0.01

6 `/api/voice` next to the chatbot; the browser check; one real turn costed

weeks/07/lab.md · about 60 minutes plus recording · in your portfolio repository · tag week07-submitted · due November 9

Before November 9

- **Week 7 submission:** portfolio repository URL + `week07-submitted`.
- **SDLC Playbook certificate** on Moodle (assigned October 26).
- Portfolio: the chatbot milestone was today; the spoken interface is this lab; **deployed** is November 30, with the hosting decision in `spec.md`.
- Week 8 is tool use and function calling, and **Team Project 2** is assigned.

Before you leave

- Why does the speech reply carry no bill, and where is the bill?
- What did the second transcriber tell us that `usage.seconds` could not?
- What does one voice turn cost, which of the three calls is most of it, and whose decision does that number belong to?

Sources and next steps

- OpenRouter audio APIs · transcription guide · text-to-speech tutorial · openrouter on PyPI.
- MediaRecorder · Cloudflare Pages Functions · Vercel environment variables · here.now docs.
- Transcription, speech, and the loop rehearsed October 26, 2026 on `openai/whisper-large-v3-turbo`, `elevenlabs/scribe-v2`, `microsoft/mai-voice-2.1-flash`, `elevenlabs/eleven-v4-turbo`, `x-ai/grok-voice-tts-1.0`, `hexgrad/kokoro-82m`, `mistralai/voxtral-mini-tts-3-0`, and `minimax/minimax-m3`; prices quoted are those runs' `usage.cost` or the catalog price times characters.

Prepared October 26, 2026.